

建议保持入参风格一致

:::caution 实验功能 Python UDF/UDAF/UDTF 是实验功能，从 Apache Doris 4.1.3 版本开始发布。 :::

Python UDF/UDAF/UDTF 是 Apache Doris 提供的自定义函数扩展机制，允许用户使用 Python 语言编写标量函数、聚合函数和表函数，用于在 SQL 中完成内置函数难以表达的复杂计算逻辑，并复用 Python 丰富的生态库。

本文从典型用户场景出发，分别介绍三类函数的用法、参数、数据类型映射、性能建议、限制以及多版本 Python 环境的部署方式。

何时选择 Python UDF/UDAF/UDTF

- 你的场景：需要按行进行复杂转换、清洗、脱敏、校验；推荐使用：Python UDF（标量函数）；关系：一行输入 → 一行输出
- 你的场景：需要在 GROUP BY 或窗口（OVER 子句）上自定义聚合指标；推荐使用：Python UDAF（聚合函数）；关系：多行输入 → 一行输出
- 你的场景：需要将一行数据展开为多行，例如 CSV/JSON 拆解、序列生成；推荐使用：Python UDTF（表函数）；关系：一行输入 → 零行/多行输出

如对性能要求极高，建议优先使用 Doris 内置（C++ 实现的）函数；Python UDF 适合内置函数无法满足、且数据量适中的场景。

通用前置条件

在创建任何 Python UDF/UDAF/UDTF 之前，请先完成以下准备：

1. **启用 Python UDF 功能并配置好 Python 环境**：在 BE 节点的 be.conf 中开启相关参数，并配置 Conda 或 venv 形式的 Python 多版本环境。详见 Python UDF/UDAF/UDTF 环境配置与多版本管理。
2. **强制依赖**：必须在所有 BE 节点对应的 Python 环境中预装 **pandas** 与 **pyarrow**，这是 Doris Python UDF 功能的强制依赖，未安装将无法运行。
3. **运行日志**：Python UDF Server 的运行日志位于 output/be/log/python_udf_output.log，可在该日志中查看函数执行情况和报错信息，便于调试。

:::tip 提示 所有创建语句都必须显式指定 runtime_version，并填写完整版本号（如 "3.10.12"），不能只写主次版本号（如 "3.10"），否则函数调用时会报错。 :::

Python UDF（标量函数）

Python UDF（User Defined Function）用于按行处理数据，每行调用一次函数并返回单个结果。它支持两种执行模式：

- **标量模式（Scalar Mode）**：逐行处理数据，适合简单的转换和计算。
- **向量化模式（Vectorized Mode）**：批量处理数据，借助 Pandas 进行高性能计算。

创建 Python UDF

Python UDF 支持两种创建方式：**内联模式（Inline）** 与 **模块模式（Module）**。

:::caution 注意 若同时指定了 file 参数和 AS \$\$ 内联 Python 代码，Doris 将**优先加载内联 Python 代码**，按内联模式运行。:::

内联模式（Inline Mode）

内联模式允许直接在 SQL 中编写 Python 代码，适合简单逻辑。

语法：

```
CREATE FUNCTION function_name(parameter_type1, parameter_type2, ...)
RETURNS return_type
PROPERTIES (
  "type" = "PYTHON_UDF",
  "symbol" = "entry_function_name",
  "runtime_version" = "python_version",
  "always_nullable" = "true|false",
  "volatility" = "immutable|stable|volatile"
)
AS $$
def entry_function_name(param1, param2, ...):
    # Python code here
    return result
$$;
```

示例 1：整数加法

```
DROP FUNCTION IF EXISTS py_add(INT, INT);
```

```
CREATE FUNCTION py_add(INT, INT)
RETURNS INT
```

```

PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "evaluate",
    "runtime_version" = "3.10.12",
    "volatility" = "immutable"
)
AS $$
def evaluate(a, b):
    return a + b
$$;

SELECT py_add(10, 20) AS result; -- 结果: 30

```

示例 2：字符串拼接（含 NULL 处理）

```

DROP FUNCTION IF EXISTS py_concat(String, String);

CREATE FUNCTION py_concat(String, String)
RETURNS String
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "evaluate",
    "runtime_version" = "3.10.12",
    "volatility" = "immutable"
)
AS $$
def evaluate(s1, s2):
    if s1 is None or s2 is None:
        return None
    return s1 + s2
$$;

SELECT py_concat('Hello', ' World') AS result; -- 结果: Hello World
SELECT py_concat(NULL, ' World') AS result; -- 结果: NULL
SELECT py_concat('Hello', NULL) AS result; -- 结果: NULL

```

模块模式 (Module Mode)

模块模式适合复杂逻辑：将 Python 代码打包成 .zip 压缩包，在创建函数时通过 file 参数引用。

步骤 1: 编写 Python 模块

创建 `python_udf_scalar_ops.py` 文件:

```
def add_three_numbers(a, b, c):  
    """Add three numbers"""  
    if a is None or b is None or c is None:  
        return None  
    return a + b + c
```

```
def reverse_string(s):  
    """Reverse a string"""  
    if s is None:  
        return None  
    return s[::-1]
```

```
def is_prime(n):  
    """Check if a number is prime"""  
    if n is None or n **注意**:
```

> - 若 ``symbol`` 格式不合法（如缺少函数名、模块名为空、路径中存在空组件等），Doris 将在函数调用时报错。

> - ZIP 包内的目录结构必须与 ``symbol`` 指定的路径一致。

> - 每个包目录下都需要包含 ``__init__.py`` 文件（可以为空）。

****步骤 5: 创建 UDF 函数****

示例 1: 使用本地文件（无包结构）

```
` `` `sql  
DROP FUNCTION IF EXISTS py_add_three(INT, INT, INT);  
DROP FUNCTION IF EXISTS py_reverse(String);  
DROP FUNCTION IF EXISTS py_is_prime(INT);  
  
CREATE FUNCTION py_add_three(INT, INT, INT)  
RETURNS INT  
PROPERTIES (  
    "type" = "PYTHON_UDF",  
    "file" = "file:///path/to/python_udf_scalar_ops.zip",  
    "symbol" = "python_udf_scalar_ops.add_three_numbers",  
    "runtime_version" = "3.10.12",
```

```

    "always_nullable" = "true",
    "volatility" = "immutable"
);

CREATE FUNCTION py_reverse(String)
RETURNS STRING
PROPERTIES (
    "type" = "PYTHON_UDF",
    "file" = "file:///path/to/python_udf_scalar_ops.zip",
    "symbol" = "python_udf_scalar_ops.reverse_string",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
);

```

```

CREATE FUNCTION py_is_prime(INT)
RETURNS BOOLEAN
PROPERTIES (
    "type" = "PYTHON_UDF",
    "file" = "file:///path/to/python_udf_scalar_ops.zip",
    "symbol" = "python_udf_scalar_ops.is_prime",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
);

```

示例 2：使用 HTTP/HTTPS 远程文件

```

DROP FUNCTION IF EXISTS py_add_three(INT, INT, INT);
DROP FUNCTION IF EXISTS py_reverse(String);
DROP FUNCTION IF EXISTS py_is_prime(INT);

```

```

CREATE FUNCTION py_add_three(INT, INT, INT)
RETURNS INT
PROPERTIES (
    "type" = "PYTHON_UDF",
    "file" = "https://your-storage.com/udf/python_udf_scalar_ops.zip",
    "symbol" = "python_udf_scalar_ops.add_three_numbers",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
);

```

```
);
```

```
CREATE FUNCTION py_reverse(STRING)
```

```
RETURNS STRING
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
```

```
    "file" = "https://your-storage.com/udf/python_udf_scalar_ops.zip",
```

```
    "symbol" = "python_udf_scalar_ops.reverse_string",
```

```
    "runtime_version" = "3.10.12",
```

```
    "always_nullable" = "true",
```

```
    "volatility" = "immutable"
```

```
);
```

```
CREATE FUNCTION py_is_prime(INT)
```

```
RETURNS BOOLEAN
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
```

```
    "file" = "https://your-storage.com/udf/python_udf_scalar_ops.zip",
```

```
    "symbol" = "python_udf_scalar_ops.is_prime",
```

```
    "runtime_version" = "3.10.12",
```

```
    "always_nullable" = "true",
```

```
    "volatility" = "immutable"
```

```
);
```

示例 3：使用包结构

```
DROP FUNCTION IF EXISTS py_multiply(INT);
```

-- ZIP 结构: my_udf/__init__.py, my_udf/math_ops.py

```
CREATE FUNCTION py_multiply(INT)
```

```
RETURNS INT
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
```

```
    "file" = "file:///path/to/my_udf.zip",
```

```
    "symbol" = "my_udf.math_ops.multiply_by_two",
```

```
    "runtime_version" = "3.10.12",
```

```
    "always_nullable" = "true",
```

```
    "volatility" = "immutable"
```

```
);
```

步骤 6：使用函数

```
SELECT py_add_three(10, 20, 30) AS sum_result; -- 结果: 60
SELECT py_reverse('hello') AS reversed;      -- 结果: olleh
SELECT py_is_prime(17) AS is_prime;          -- 结果: true
```

删除 Python UDF

-- 语法

```
DROP FUNCTION IF EXISTS function_name(parameter_type1, parameter_type2, ...);
```

-- 示例

```
DROP FUNCTION IF EXISTS py_add_three(INT, INT, INT);
DROP FUNCTION IF EXISTS py_reverse(STRING);
DROP FUNCTION IF EXISTS py_is_prime(INT);
```

参数说明

CREATE FUNCTION 参数

- 参数：function_name；是否必需：是；说明：函数名称，需要符合标识符命名规则
- 参数：parameter_type；是否必需：是；说明：参数类型列表，支持 Doris 的各种数据类型
- 参数：return_type；是否必需：是；说明：返回值类型

PROPERTIES 参数

- 参数：type；是否必需：是；默认值：-；说明：固定为 "PYTHON_UDF"
- 参数：symbol；是否必需：是；默认值：-；说明：Python 函数入口名称。
 - **内联模式**：直接写函数名，如 "evaluate"
 - **模块模式**：格式为 [package_name.]module_name.func_name，详见模块模式说明
- 参数：file；是否必需：否；默认值：-；说明：Python .zip 包路径，仅模块模式需要。支持三种协议：
 - file:// —— 本地文件系统路径
 - http:// —— HTTP 远程下载
 - https:// —— HTTPS 远程下载
- 参数：runtime_version；是否必需：是；默认值：-；说明：Python 运行时版本，如 "3.10.12"，需填写完整的版本号
- 参数：always_nullable；是否必需：否；默认值：true；说明：是否总是返回可空结果
- 参数：volatility；是否必需：否；默认值：volatile；说明：函数易变性。可选值为 immutable、stable、volatile。immutable：相同输入始终产生相同输出。大部分确定性 UDF 建议设置为 immutable，这样计划优化效果更好。stable：在同一条 SQL 语句内结果稳定，但不同语句之间可能变化，类似 now()。这类函数会禁用

SQL Cache 和物化视图改写。volatile：每次调用结果都可能变化，类似 random()。这类函数会禁用 SQL Cache、物化视图改写以及很多优化器改写规则。

运行时版本说明

- 支持 Python 3.x 版本。
- 需要指定完整版本号（如 "3.10.12"），不能只填写主次版本号（如 "3.10"）。
- 如果不指定 runtime_version，函数调用时将报错。

数据类型映射

下表列出了 Doris 数据类型与 Python 类型之间的映射关系：

- 类型分类：空类型；Doris 类型：NULL；Python 类型：None；说明：空值
- 类型分类：布尔类型；Doris 类型：BOOLEAN；Python 类型：bool；说明：布尔值
- 类型分类：整数类型；Doris 类型：TINYINT；Python 类型：int；说明：8 位整数
- Doris 类型：SMALLINT；Python 类型：int；说明：16 位整数
- Doris 类型：INT；Python 类型：int；说明：32 位整数
- Doris 类型：BIGINT；Python 类型：int；说明：64 位整数
- Doris 类型：LARGEINT；Python 类型：int；说明：128 位整数
- 类型分类：浮点类型；Doris 类型：FLOAT；Python 类型：float；说明：32 位浮点数
- Doris 类型：DOUBLE；Python 类型：float；说明：64 位浮点数
- Doris 类型：TIME / TIMEV2；Python 类型：float；说明：时间类型（以浮点数表示）
- 类型分类：字符串类型；Doris 类型：CHAR；Python 类型：str；说明：定长字符串
- Doris 类型：VARCHAR；Python 类型：str；说明：变长字符串
- Doris 类型：STRING；Python 类型：str；说明：字符串
- Doris 类型：JSONB；Python 类型：str；说明：JSON 二进制格式（转换为字符串）
- Doris 类型：VARIANT；Python 类型：str；说明：变体类型（转换为字符串）
- Doris 类型：DATE；Python 类型：str；说明：日期字符串，格式为 'YYYY-MM-DD'
- Doris 类型：DATETIME；Python 类型：str；说明：日期时间字符串，格式为 'YYYY-MM-DD HH:MM:SS'

- 类型分类：日期时间类型；Doris 类型：DATEV2；Python 类型：datetime.date；说明：日期对象
- Doris 类型：DATETIMEV2；Python 类型：datetime.datetime；说明：日期时间对象
- Doris 类型：TIMESTAMPV2；Python 类型：datetime.datetime；说明：带时区的日期时间对象
- 类型分类：Decimal 类型；Doris 类型：DECIMAL / DECIMALV2；Python 类型：decimal.Decimal；说明：高精度小数
- Doris 类型：DECIMAL32；Python 类型：decimal.Decimal；说明：32 位定点数
- Doris 类型：DECIMAL64；Python 类型：decimal.Decimal；说明：64 位定点数
- Doris 类型：DECIMAL128；Python 类型：decimal.Decimal；说明：128 位定点数
- Doris 类型：DECIMAL256；Python 类型：decimal.Decimal；说明：256 位定点数
- 类型分类：IP 类型；Doris 类型：IPV4；Python 类型：ipaddress.IPv4Address；说明：IPv4 地址
- Doris 类型：IPV6；Python 类型：ipaddress.IPv6Address；说明：IPv6 地址
- 类型分类：二进制类型；Doris 类型：BITMAP；Python 类型：bytes；说明：位图数据（暂不支持该类型）
- Doris 类型：HLL；Python 类型：bytes；说明：HyperLogLog 数据（暂不支持该类型）
- Doris 类型：QUANTILE_STATE；Python 类型：bytes；说明：分位数状态数据（暂不支持该类型）
- 类型分类：复杂数据类型；Doris 类型：ARRAY；Python 类型：list；说明：数组，元素类型为 T
- Doris 类型：MAP；Python 类型：dict；说明：字典，键类型为 K，值类型为 V
- Doris 类型：STRUCT；Python 类型：dict；说明：结构体，字段名为键，字段值为值

NULL 值处理

- Doris 的 NULL 值在 Python 中映射为 None。
- 如果函数参数为 NULL，Python 函数接收到的是 None。
- 如果 Python 函数返回 None，Doris 将其视为 NULL。
- 建议在函数中显式处理 None 值，避免运行时错误。

示例：

```
DROP FUNCTION IF EXISTS py_safe_divide(DOUBLE, DOUBLE);
```

```
CREATE FUNCTION py_safe_divide(DOUBLE, DOUBLE)
```

```
RETURNS DOUBLE
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",  
    "symbol" = "evaluate",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def evaluate(a, b):
```

```
    if a is None or b is None:
```

```
        return None
```

```
    if b == 0:
```

```
        return None
```

```
    return a / b
```

```
$$;
```

```
SELECT py_safe_divide(10.0, 2.0); -- 结果: 5.0
```

```
SELECT py_safe_divide(10.0, 0.0); -- 结果: NULL
```

```
SELECT py_safe_divide(10.0, NULL); -- 结果: NULL
```

向量化模式

向量化模式使用 Pandas 批量处理数据，性能优于标量模式。在向量化模式下，函数参数为 pandas.Series 对象，返回值也应为 pandas.Series。

:::caution 注意 为确保系统正确识别向量化模式，请在函数签名中使用类型注解（如 a: pd.Series），并在函数逻辑中直接操作批量数据结构。若未明确使用向量化类型，系统将回退到标量模式（Scalar Mode）。

若函数签名同时包含 pd.Series 类型与普通类型参数，系统会将普通类型参数对应的输入列按常量列处理（即整批数据复用同一个值），可能导致结果与预期不一致。建议在向量化模式下保持入参风格一致：要么全部使用 pandas.Series 类型注解，要么全部使用普通类型参数（标量模式）。 :::

```
## 向量化模式
```

```
def add(a: pd.Series, b: pd.Series) -> pd.Series:
```

```
    return a + b + 1
```

标量模式

```
def add(a, b):  
    return a + b + 1
```

基本示例

示例 1：向量化整数加法

```
DROP FUNCTION IF EXISTS py_vec_add(INT, INT);
```

```
CREATE FUNCTION py_vec_add(INT, INT)
```

```
RETURNS INT
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",  
    "symbol" = "add",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def add(a: pd.Series, b: pd.Series) -> pd.Series:
```

```
    return a + b + 1
```

```
$$;
```

```
SELECT py_vec_add(1, 2); -- 结果: 4
```

示例 2：向量化字符串处理

```
DROP FUNCTION IF EXISTS py_vec_upper(String);
```

```
CREATE FUNCTION py_vec_upper(String)
```

```
RETURNS String
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",  
    "symbol" = "to_upper",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"
```

```
)
```

AS \$\$

```
def to_upper(s: pd.Series) -> pd.Series:  
    return s.str.upper()  
$$;
```

SELECT py_vec_upper('hello'); -- 结果: 'HELLO'

示例 3：向量化数学运算

DROP FUNCTION IF EXISTS py_vec_sqrt(DOUBLE);

```
CREATE FUNCTION py_vec_sqrt(DOUBLE)  
RETURNS DOUBLE  
PROPERTIES (  
    "type" = "PYTHON_UDF",  
    "symbol" = "sqrt",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"  
)  
AS $$
```

```
def sqrt(x: pd.Series) -> pd.Series:  
    return np.sqrt(x)  
$$;
```

SELECT py_vec_sqrt(16); -- 结果: 4.0

示例 4：函数签名参数类型不一致（同时包含 pd.Series 类型与普通类型）

```
CREATE TABLE t_bug_013 (  
    id INT,  
    a INT,  
    b INT  
) ENGINE=OLAP  
DUPLICATE KEY(id)  
DISTRIBUTED BY HASH(id) BUCKETS 1  
PROPERTIES ("replication_num" = "1");  
  
INSERT INTO t_bug_013 VALUES
```

```
(1, 1, 10),
(2, 2, 20),
(3, 3, 30),
(4, 4, NULL),
(5, NULL, 50);
```

```
DROP FUNCTION IF EXISTS py_mixed_vector_add(INT, INT);
```

```
CREATE FUNCTION py_mixed_vector_add(INT, INT)
RETURNS INT
PROPERTIES (
  "type"="PYTHON_UDF",
  "symbol"="py_mixed_vector_add_impl",
  "always_nullable"="true",
  "runtime_version"="3.12.11",
  "volatility"="immutable"
)
AS $$
```

```
# 建议保持入参风格一致
```

```
def py_mixed_vector_add_impl(x: pd.Series, y: int):
    return x + y
$$;
```

```
SELECT
  id,
  a,
  b,
  py_mixed_vector_add(a, b) AS vector_val
FROM t_bug_013
ORDER BY id;
```

```
-- 将 b 列视为常量列处理
```

```
+-----+-----+-----+-----+
```

```
| id | a | b | vector_val |
```

```
+-----+-----+-----+-----+
```

```
| 1 | 1 | 10 | 11 |
| 2 | 2 | 20 | 12 |
| 3 | 3 | 30 | 13 |
| 4 | 4 | NULL | 14 |
```

```
| 5 | NULL | 50 | NULL |
+-----+-----+-----+-----+
```

向量化模式的优势

1. **性能优化**：批量处理数据，减少 Python 与 Doris 之间的交互次数。
2. **利用 Pandas/NumPy**：充分发挥向量化计算的性能优势。
3. **简洁代码**：使用 Pandas API 可以更简洁地表达复杂逻辑。

使用向量化函数

```
DROP TABLE IF EXISTS test_table;
```

```
CREATE TABLE test_table (
  id INT,
  value INT,
  text STRING,
  score DOUBLE
) ENGINE=OLAP
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES("replication_num" = "1");
```

```
INSERT INTO test_table VALUES
```

```
(1, 10, 'hello', 85.5),
(2, 20, 'world', 92.0),
(3, 30, 'python', 78.3);
```

```
SELECT
```

```
  id,
  py_vec_add(value, value) AS sum_result,
  py_vec_upper(text) AS upper_text,
  py_vec_sqrt(score) AS sqrt_score
FROM test_table;
```

```
+-----+-----+-----+-----+
| id | sum_result | upper_text | sqrt_score |
+-----+-----+-----+-----+
| 1 | 21 | HELLO | 9.246621004453464 |
| 2 | 41 | WORLD | 9.591663046625438 |
| 3 | 61 | PYTHON | 8.848728722251575 |
+-----+-----+-----+-----+
```

复杂数据类型处理

ARRAY 类型

示例：数组元素求和

```
DROP FUNCTION IF EXISTS py_array_sum(ARRAY);
```

```
CREATE FUNCTION py_array_sum(ARRAY)
```

```
RETURNS INT
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",  
    "symbol" = "evaluate",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def evaluate(arr):
```

```
    """ Doris 中的 ARRAY 类型对应 Python 中的 list """
```

```
    if arr is None:
```

```
        return None
```

```
    return sum(arr)
```

```
$$;
```

```
SELECT py_array_sum([1, 2, 3, 4, 5]) AS result; -- 结果: 15
```

示例：数组过滤

```
DROP FUNCTION IF EXISTS py_array_filter_positive(ARRAY);
```

```
CREATE FUNCTION py_array_filter_positive(ARRAY)
```

```
RETURNS ARRAY
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",  
    "symbol" = "evaluate",  
    "runtime_version" = "3.10.12",  
    "always_nullable" = "true",  
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def evaluate(arr):
```

```

    if arr is None:
        return None
    return [x for x in arr if x > 0]
$$;

```

```

SELECT py_array_filter_positive([1, -2, 3, -4, 5]) AS result; -- 结果: [1, 3, 5]

```

MAP 类型

示例：获取 MAP 的键数量

```

DROP FUNCTION IF EXISTS py_map_size(MAP);

```

```

CREATE FUNCTION py_map_size(MAP)
RETURNS INT
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "evaluate",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
)
AS $$
def evaluate(m):
    """ Doris 中的 MAP 类型对应 Python 中的 dict """
    if m is None:
        return None
    return len(m)
$$;

```

```

SELECT py_map_size({'a': 1, 'b': 2, 'c': 3}) AS result; -- 结果: 3

```

示例：获取 MAP 中的值

```

DROP FUNCTION IF EXISTS py_map_get(MAP, STRING);

```

```

CREATE FUNCTION py_map_get(MAP, STRING)
RETURNS STRING
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "evaluate",
    "runtime_version" = "3.10.12",

```



```

    "always_nullable" = "true",
    "volatility" = "immutable"
)
AS $$
def evaluate(m, key):
    if m is None or key is None:
        return None
    return m.get(key)
$$;

```

SELECT py_map_get({'name': 'Alice', 'age': '30'}, 'name') **AS** result; -- 结果: Alice

STRUCT 类型

示例：访问 STRUCT 字段

DROP FUNCTION IF EXISTS py_struct_get_name(STRUCT);

CREATE FUNCTION py_struct_get_name(STRUCT)

RETURNS STRING

PROPERTIES (

```

    "type" = "PYTHON_UDF",
    "symbol" = "evaluate",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"

```

)

AS \$\$

def evaluate(s):

""" Doris 中的 STRUCT 类型对应 Python 中的 dict """

if s **is** None:

return None

return s.get('name')

\$\$;

SELECT py_struct_get_name({'Alice', 30}) **AS** result; -- 结果: Alice

实际应用场景

场景 1：数据脱敏

DROP FUNCTION IF EXISTS py_mask_email(String);

CREATE FUNCTION py_mask_email(String)

RETURNS String

Properties (

 "type" = "PYTHON_UDF",
 "symbol" = "evaluate",
 "runtime_version" = "3.10.12",
 "volatility" = "immutable"

)

AS \$\$

def evaluate(email):

if email **is None** or '@' **not in** email:
 return None

 parts = email.split('@')

if len(parts[0]) > 0:

return parts[0] * 2

2. 使用模块模式管理复杂逻辑

将复杂的函数逻辑放在独立的 Python 文件中，便于维护和复用。

3. 避免在函数中执行 I/O 操作

不建议在 UDF 中进行文件读写、网络请求等 I/O 操作，这会严重影响性能。

限制与注意事项

1. Python 版本支持

- 仅支持 Python 3.x 版本。
- 建议使用 Python 3.10 或更高版本。
- 确保 Doris 集群已安装对应的 Python 运行时。

2. 依赖库

- 内置支持 Python 标准库。
- 如需使用第三方库，需要在集群环境中预先安装。

3. 性能考虑

- Python UDF 性能低于 Doris 内置函数（C++ 实现）。
- 对于性能敏感场景，建议优先考虑 Doris 内置函数。
- 大数据量场景建议使用向量化模式。

4. 安全性

- UDF 代码在 Doris 进程中执行，需要确保代码安全可信。
- 避免在 UDF 中执行危险操作（如系统命令、文件删除等）。
- 生产环境建议对 UDF 代码进行审核。

5. 资源限制

- UDF 执行会占用 BE 节点的 CPU 和内存资源。
- 大量使用 UDF 可能影响集群整体性能。
- 建议监控 UDF 的资源消耗情况。

常见问题

Q1: 如何在 Python UDF 中使用第三方库？

A: 需要在所有 BE 节点上安装对应的 Python 库。例如：

```
pip3 install numpy pandas  
conda install numpy pandas
```

Q2: Python UDF 是否支持递归函数？

A: 支持，但需要注意递归深度，避免栈溢出。

Q3: 如何调试 Python UDF？

A: 可以在本地 Python 环境中先调试函数逻辑，确保无误后再创建 UDF。可以查看 BE 日志获取错误信息。

Q4: Python UDF 是否支持全局变量？

A: 支持，但不建议使用全局变量，因为在分布式环境中全局变量的行为可能不符合预期。

Q5: 如何更新已存在的 Python UDF？

A: 先删除旧的 UDF，再创建新的：

```
DROP FUNCTION IF EXISTS function_name(parameter_types);  
CREATE FUNCTION function_name(...) ...;
```

Q6: Python UDF 能否访问外部资源?

A: 技术上可以，但**强烈不建议**。Python UDF 中可以使用网络请求库（如 requests）访问外部 API、数据库等，但这会严重影响性能和稳定性。原因包括：

- 网络延迟会导致查询变慢。
- 外部服务不可用时会导致 UDF 失败。
- 大量并发请求可能造成外部服务压力。
- 难以控制超时和错误处理。

Python UDAF（聚合函数）

Python UDAF（User Defined Aggregate Function）允许用户自定义聚合函数，用于数据分组聚合和窗口计算。通过 Python UDAF，可以灵活实现复杂的聚合逻辑，如统计分析、数据收集、自定义指标计算等。

Python UDAF 的核心特点：

- **分布式聚合**：支持分布式环境下的数据聚合，自动处理数据的分区、合并和最终计算。
- **状态管理**：通过类实例维护聚合状态，支持复杂的状态对象。
- **窗口函数支持**：可用于窗口函数（OVER 子句），实现移动聚合、排名等高级功能。
- **灵活性强**：可实现任意复杂的聚合逻辑，不受内置聚合函数限制。

UDAF 基本概念

聚合函数的生命周期

Python UDAF 通过类来实现，一个聚合函数的执行包含以下阶段：

1. **初始化（__init__）**：创建聚合状态对象，初始化状态变量。
2. **累积（accumulate）**：处理单行数据，更新聚合状态。
3. **合并（merge）**：合并多个分区的聚合状态（分布式场景）。
4. **完成（finish）**：计算并返回最终聚合结果。

必需的类方法和属性

一个完整的 Python UDAF 类必须实现以下方法：

- 方法/属性：__init__(self)；说明：初始化聚合状态；是否必需：是
- 方法/属性：accumulate(self, *args)；说明：累积单行数据；是否必需：是
- 方法/属性：merge(self, other_state)；说明：合并其他分区的状态；是否必需：是
- 方法/属性：finish(self)；说明：返回最终聚合结果；是否必需：是
- 方法/属性：aggregate_state（属性）；说明：返回可序列化的聚合状态，**必须支持 pickle 序列化**；是否必需：是

基本语法

创建 Python UDAF

Python UDAF 支持两种创建方式：**内联模式（Inline）** 与 **模块模式（Module）**。

:::tip 注意 若同时指定了 file 参数和 AS \$\$ 内联 Python 代码，Doris 将**优先加载内联 Python 代码**，按内联模式运行 Python UDAF。:::

内联模式（Inline Mode）

内联模式允许直接在 SQL 中编写 Python 类，适合简单的聚合逻辑。

语法：

```
CREATE AGGREGATE FUNCTION function_name(parameter_type1, parameter_type2, ...)
RETURNS return_type
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "ClassName",
    "runtime_version" = "python_version",
    "always_nullable" = "true|false"
)
AS $$
class ClassName:
    def __init__(self):
        # 初始化状态变量

    @property
    def aggregate_state(self):
        # 返回可序列化的状态

    def accumulate(self, *args):
        # 累积数据
```

```
def merge(self, other_state):
    # 合并状态

def finish(self):
    # 返回最终结果
$$;
```

示例 1：求和聚合

```
DROP TABLE IF EXISTS sales;
```

```
CREATE TABLE IF NOT EXISTS sales (
    id INT,
    category VARCHAR(50),
    amount INT
) DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES("replication_num" = "1");
```

```
INSERT INTO sales VALUES
```

```
(1, 'Electronics', 1000),
(2, 'Electronics', 1500),
(3, 'Books', 200),
(4, 'Books', 300),
(5, 'Clothing', 500),
(6, 'Clothing', 800),
(7, 'Electronics', 2000),
(8, 'Books', 150);
```

```
DROP FUNCTION IF EXISTS py_sum(INT);
```

```
CREATE AGGREGATE FUNCTION py_sum(INT)
RETURNS BIGINT
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "SumUDAF",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```

)
AS $$
class SumUDAF:
    def __init__(self):
        self.total = 0

    @property
    def aggregate_state(self):
        return self.total

    def accumulate(self, value):
        if value is not None:
            self.total += value

    def merge(self, other_state):
        self.total += other_state

    def finish(self):
        return self.total
$$;

SELECT category, py_sum(amount) as total_amount
FROM sales
GROUP BY category
ORDER BY category;

```

```

+-----+-----+
| category | total_amount |
+-----+-----+
| Books    | 650          |
| Clothing | 1300         |
| Electronics | 4500        |
+-----+-----+

```

示例 2：平均值聚合

```

DROP TABLE IF EXISTS employees;

CREATE TABLE IF NOT EXISTS employees (
    id INT,

```

```
name VARCHAR(100),
department VARCHAR(50),
salary DOUBLE
) DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES("replication_num" = "1");
```

```
INSERT INTO employees VALUES
(1, 'Alice', 'Engineering', 80000.0),
(2, 'Bob', 'Engineering', 90000.0),
(3, 'Charlie', 'Sales', 60000.0),
(4, 'David', 'Sales', 80000.0),
(5, 'Eve', 'HR', 50000.0),
(6, 'Frank', 'Engineering', 70000.0),
(7, 'Grace', 'HR', 70000.0);
```

```
DROP FUNCTION IF EXISTS py_avg(DOUBLE);
```

```
CREATE AGGREGATE FUNCTION py_avg(DOUBLE)
RETURNS DOUBLE
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "AvgUDAF",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
)
```

```
AS $$
```

```
class AvgUDAF:
```

```
    def __init__(self):
        self.sum = 0.0
        self.count = 0
```

```
    @property
    def aggregate_state(self):
        return (self.sum, self.count)
```

```
    def accumulate(self, value):
        if value is not None:
            self.sum += value
```



```
self.count += 1
```

```
def merge(self, other_state):  
    other_sum, other_count = other_state  
    self.sum += other_sum  
    self.count += other_count
```

```
def finish(self):  
    if self.count == 0:  
        return None  
    return self.sum / self.count
```

\$\$;

```
SELECT department, py_avg(salary) as avg_salary  
FROM employees  
GROUP BY department  
ORDER BY department;
```

```
+-----+-----+  
| department | avg_salary |  
+-----+-----+  
| Engineering | 80000 |  
| HR          | 60000 |  
| Sales       | 70000 |  
+-----+-----+
```

模块模式 (Module Mode)

模块模式适合复杂的聚合逻辑，需要将 Python 代码打包成 .zip 压缩包，并在函数创建时引用。

步骤 1：编写 Python 模块

创建 stats_udaf.py 文件：

```
class VarianceUDAF:  
    """计算总体方差"""  
  
    def __init__(self):  
        self.count = 0
```

```
self.sum_val = 0.0
self.sum_sq = 0.0
```

@property

```
def aggregate_state(self):
    return (self.count, self.sum_val, self.sum_sq)
```

```
def accumulate(self, value):
    if value is not None:
        self.count += 1
        self.sum_val += value
        self.sum_sq += value * value
```

```
def merge(self, other_state):
    other_count, other_sum, other_sum_sq = other_state
    self.count += other_count
    self.sum_val += other_sum
    self.sum_sq += other_sum_sq
```

```
def finish(self):
    if self.count == 0:
        return None
    mean = self.sum_val / self.count
    variance = (self.sum_sq / self.count) - (mean * mean)
    return variance
```

class StdDevUDAF:

"""计算总体标准差"""

```
def __init__(self):
    self.count = 0
    self.sum_val = 0.0
    self.sum_sq = 0.0
```

@property

```
def aggregate_state(self):
    return (self.count, self.sum_val, self.sum_sq)
```

```
def accumulate(self, value):
```

```
if value is not None:
    self.count += 1
    self.sum_val += value
    self.sum_sq += value * value
```

```
def merge(self, other_state):
    other_count, other_sum, other_sum_sq = other_state
    self.count += other_count
    self.sum_val += other_sum
    self.sum_sq += other_sum_sq
```

```
def finish(self):
    if self.count == 0:
        return None
    mean = self.sum_val / self.count
    variance = (self.sum_sq / self.count) - (mean * mean)
    return math.sqrt(max(0, variance))
```

```
class MedianUDAF:
```

```
    """计算中位数"""
```

```
def __init__(self):
    self.values = []
```

```
@property
```

```
def aggregate_state(self):
    return self.values
```

```
def accumulate(self, value):
    if value is not None:
        self.values.append(value)
```

```
def merge(self, other_state):
    if other_state:
        self.values.extend(other_state)
```

```
def finish(self):
    if not self.values:
        return None
```

```

sorted_vals = sorted(self.values)
n = len(sorted_vals)
if n % 2 == 0:
    return (sorted_vals[n//2 - 1] + sorted_vals[n//2]) / 2.0
else:
    return sorted_vals[n//2]

```

步骤 2：打包 Python 模块

必须将 Python 文件打包成 .zip 格式（即使只有单个文件）：

```
zip stats_udaf.zip stats_udaf.py
```

步骤 3：设置 .zip 包的路径

通过 file 参数指定 .zip 包的路径：

- 部署方式：本地文件系统（file:// 协议）；写法："file" = "file:///path/to/stats_udaf.zip"
- 部署方式：HTTP/HTTPS 远程下载（http:// 或 https:// 协议）；写法："file" = "http://example.com/udaf/stats_udaf.zip" 或 "file" = "https://s3.amazonaws.com/bucket/stats_udaf.zip"

注意：

- 使用远程下载方式时，需确保所有 BE 节点都能访问该 URL。
- 首次调用时会下载文件，可能有一定延迟。
- 文件会被缓存，后续调用无需重复下载。
- 使用 file:// 时，Doris 会在 CREATE AGGREGATE FUNCTION 阶段从 FE 节点读取该包并计算 checksum，在执行阶段从 BE 节点读取该包。同一个 .zip 文件必须以相同绝对路径存在于所有 FE 和 BE 节点，并且文件内容必须完全一致。如果文件只存在于 BE 节点，函数创建会失败；如果 FE 和 BE 上的文件内容不一致，执行时会因 checksum 不匹配失败。

步骤 4：设置 symbol 参数

在模块模式下，symbol 用于指定类在 ZIP 包中的位置，格式为：

```
[package_name.]module_name.ClassName
```

参数说明：

- package_name（可选）：ZIP 包内顶层 Python 包的名称。
- module_name（必填）：包含目标类的 Python 模块文件名（不含 .py 后缀）。

- ClassName（必填）：UDAF 类名。

解析规则：

- Doris 会将 symbol 字符串按 . 分割：
 - 如果得到**两个**子字符串，分别为 module_name 和 ClassName。
 - 如果得到**三个及以上**的子字符串，开头为 package_name，中间为 module_name，结尾为 ClassName。

:::caution Warning 命名空间应具备唯一性，避免与 Python 标准库或常用第三方库同名，以避免因模块遮蔽导致的依赖冲突及运行时异常。 :::

步骤 5：创建 UDAF 函数

```
DROP FUNCTION IF EXISTS py_variance(DOUBLE);
DROP FUNCTION IF EXISTS py_stddev(DOUBLE);
DROP FUNCTION IF EXISTS py_median(DOUBLE);
```

```
CREATE AGGREGATE FUNCTION py_variance(DOUBLE)
RETURNS DOUBLE
PROPERTIES (
  "type" = "PYTHON_UDF",
  "file" = "file:///path/to/stats_udaf.zip",
  "symbol" = "stats_udaf.VarianceUDAF",
  "runtime_version" = "3.10.12",
  "always_nullable" = "true",
  "volatility" = "immutable"
);
```

```
CREATE AGGREGATE FUNCTION py_stddev(DOUBLE)
RETURNS DOUBLE
PROPERTIES (
  "type" = "PYTHON_UDF",
  "file" = "file:///path/to/stats_udaf.zip",
  "symbol" = "stats_udaf.StdDevUDAF",
  "runtime_version" = "3.10.12",
  "always_nullable" = "true",
  "volatility" = "immutable"
);
```

```
CREATE AGGREGATE FUNCTION py_median(DOUBLE)
```

```
RETURNS DOUBLE
PROPERTIES (
  "type" = "PYTHON_UDF",
  "file" = "file:///path/to/stats_udaf.zip",
  "symbol" = "stats_udaf.MedianUDAF",
  "runtime_version" = "3.10.12",
  "always_nullable" = "true",
  "volatility" = "immutable"
);
```

步骤 6：使用函数

```
DROP TABLE IF EXISTS exam_results;
```

```
CREATE TABLE IF NOT EXISTS exam_results (
  id INT,
  student_name VARCHAR(100),
  category VARCHAR(50),
  score DOUBLE
) DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES("replication_num" = "1");
```

```
INSERT INTO exam_results VALUES
```

```
(1, 'Alice', 'Math', 85.0),
(2, 'Bob', 'Math', 92.0),
(3, 'Charlie', 'Math', 78.0),
(4, 'David', 'Math', 88.0),
(5, 'Eve', 'Math', 95.0),
(6, 'Frank', 'English', 75.0),
(7, 'Grace', 'English', 82.0),
(8, 'Henry', 'English', 88.0),
(9, 'Iris', 'English', 79.0),
(10, 'Jack', 'Physics', 90.0),
(11, 'Kate', 'Physics', 85.0),
(12, 'Lily', 'Physics', 92.0),
(13, 'Mike', 'Physics', 88.0);
```

```
SELECT
```

```
  category,
  py_variance(score) as variance,
```

```

    py_stddev(score) as std_dev,
    py_median(score) as median
FROM exam_results
GROUP BY category
ORDER BY category;

```

```

+-----+-----+-----+-----+
| category | variance | std_dev | median |
+-----+-----+-----+-----+
| English | 22.5 | 4.743416490252569 | 80.5 |
| Math | 34.64000000000033 | 5.885575587824892 | 88 |
| Physics | 6.6875 | 2.58602010819715 | 89 |
+-----+-----+-----+-----+

```

删除 Python UDAF

-- 语法

```

DROP FUNCTION IF EXISTS function_name(parameter_types);

```

-- 示例

```

DROP FUNCTION IF EXISTS py_sum(INT);
DROP FUNCTION IF EXISTS py_avg(DOUBLE);
DROP FUNCTION IF EXISTS py_variance(DOUBLE);

```

参数说明

CREATE AGGREGATE FUNCTION 参数

- 参数：function_name；说明：函数名称，遵循 SQL 标识符命名规则
- 参数：parameter_types；说明：参数类型列表，如 INT、DOUBLE、STRING 等
- 参数：RETURNS return_type；说明：返回值类型

PROPERTIES 参数

- 参数：type；是否必需：是；默认值：-；说明：固定为 "PYTHON_UDF"
- 参数：symbol；是否必需：是；默认值：-；说明：Python 类名。
 - 内联模式：直接写类名，如 "SumUDAF"
 - 模块模式：格式为 [package_name.]module_name.ClassName
- 参数：file；是否必需：否；默认值：-；说明：Python .zip 包路径，仅模块模式需要。支持三种协议：
 - file:// —— 本地文件系统路径
 - http:// —— HTTP 远程下载
 - https:// —— HTTPS 远程下载

- 参数：runtime_version；是否必需：是；默认值：-；说明：Python 运行时版本，如 "3.10.12"
- 参数：always_nullable；是否必需：否；默认值：true；说明：是否总是返回可空结果
- 参数：volatility；是否必需：否；默认值：immutable；说明：函数易变性。可选值为 immutable、stable、volatile。immutable：相同输入始终产生相同输出。大部分确定性 UDF 建议设置为 immutable，这样计划优化效果更好。stable：在同一条 SQL 语句内结果稳定，但不同语句之间可能变化，类似 now()。这类函数会禁用 SQL Cache 和物化视图改写。volatile：每次调用结果都可能变化，类似 random()。这类函数会禁用 SQL Cache、物化视图改写以及很多优化器改写规则。

runtime_version 说明

- 必须填写 Python 版本的**完整版本号**，格式为 x.x.x 或 x.x.xx。
- Doris 会在配置的 Python 环境中查找匹配该版本的解释器。

窗口函数 (Window Functions)

Python UDAF 可以与窗口函数（OVER 子句）结合使用：

若将 Python UDAF 用于窗口函数（OVER 子句），Doris 会在计算每个 window frame 后调用 UDAF 的 reset 方法，需要在类中实现它以将聚合状态重置为初始值。

```
DROP TABLE IF EXISTS daily_sales_data;
```

```
CREATE TABLE IF NOT EXISTS daily_sales_data (
  sales_date DATE,
  daily_sales DOUBLE
) DUPLICATE KEY(sales_date)
DISTRIBUTED BY HASH(sales_date) BUCKETS 1
PROPERTIES("replication_num" = "1");
```

```
INSERT INTO daily_sales_data VALUES
('2024-01-01', 1000),
('2024-01-01', 800),
('2024-01-02', 1200),
('2024-01-02', 950),
('2024-01-03', 900),
('2024-01-03', 1100),
```



```
('2024-01-04', 1500),  
( '2024-01-04', 850),  
( '2024-01-05', 1100),  
( '2024-01-05', 1300);
```

```
DROP FUNCTION IF EXISTS py_running_sum(DOUBLE);
```

```
CREATE AGGREGATE FUNCTION py_running_sum(DOUBLE)
```

```
RETURNS DOUBLE
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
```

```
    "symbol" = "RunningSumUDAF",
```

```
    "runtime_version" = "3.10.12",
```

```
    "always_nullable" = "true",
```

```
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
class RunningSumUDAF:
```

```
    def __init__(self):
```

```
        self.total = 0.0
```

```
    def reset(self):
```

```
        self.total = 0.0
```

```
    @property
```

```
    def aggregate_state(self):
```

```
        return self.total
```

```
    def accumulate(self, value):
```

```
        if value is not None:
```

```
            self.total += value
```

```
    def merge(self, other_state):
```

```
        self.total += other_state
```

```
    def finish(self):
```

```
        return self.total
```

```
$$;
```

```

SELECT
  sales_date,
  daily_sales,
  py_running_sum(daily_sales) OVER (
    ORDER BY sales_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) as last_3_days_sum
FROM daily_sales_data
ORDER BY sales_date;

```

```

+-----+-----+-----+
| sales_date | daily_sales | last_3_days_sum |
+-----+-----+-----+
| 2024-01-01 |      800 |      800 |
| 2024-01-01 |     1000 |     1800 |
| 2024-01-02 |      950 |     2750 |
| 2024-01-02 |     1200 |     3150 |
| 2024-01-03 |     1100 |     3250 |
| 2024-01-03 |      900 |     3200 |
| 2024-01-04 |      850 |     2850 |
| 2024-01-04 |     1500 |     3250 |
| 2024-01-05 |     1300 |     3650 |
| 2024-01-05 |     1100 |     3900 |
+-----+-----+-----+

```

数据类型映射

Python UDAF 使用与 Python UDF 完全相同的数据类型映射规则，包括整数、浮点、字符串、日期时间、Decimal、布尔等所有类型。

详细的数据类型映射关系请参考：数据类型映射。

NULL 值处理

- Doris 会将 SQL 中的 NULL 值映射为 Python 的 None。
- 在 accumulate 方法中，需要检查参数是否为 None。
- 聚合函数可以返回 None 表示结果为 NULL。

实际应用场景

场景 1: 计算百分位数

DROP FUNCTION IF EXISTS py_percentile(DOUBLE, INT);

CREATE AGGREGATE FUNCTION py_percentile(DOUBLE, INT)

RETURNS DOUBLE

PROPERTIES (

 "type" = "PYTHON_UDF",
 "symbol" = "PercentileUDAF",
 "runtime_version" = "3.10.12",
 "always_nullable" = "true",
 "volatility" = "immutable"

)

AS \$\$

class PercentileUDAF:

 """计算百分位数，第二个参数为百分位(0-100)"""

 def __init__(self):

 self.values = []

 self.percentile = 50 # 默认中位数

 @property

 def aggregate_state(self):

 return self.values

 def accumulate(self, value, percentile):

 if value is not None:

 self.values.append(value)

 if percentile is not None:

 self.percentile = percentile

 def merge(self, other_state):

 if other_state:

 self.values.extend(other_state)

 def finish(self):

 if not self.values:

 return None

 sorted_vals = sorted(self.values)

```

n = len(sorted_vals)
k = (n - 1) * (self.percentile / 100.0)
f = int(k)
c = k - f
if f + 1 ` 定义一致。

```

输出方式

- **单列输出**：`yield value` 产出单个值。
- **多列输出**：`yield (value1, value2, ...)` 产出多个值的元组。
- **条件跳过**：不调用 `yield`，该行不产生任何输出。

基本语法

创建 Python UDTF

Python UDTF 支持两种创建方式：**内联模式 (Inline)** 与 **模块模式 (Module)**。

:::caution 注意

若同时指定了 `file` 参数和 `AS \$\$` 内联 Python 代码，Doris 将**优先加载内联 Python 代码**，按内联模式运行 Python UDTF。

:::

内联模式 (Inline Mode)

内联模式允许直接在 SQL 中编写 Python 函数，适合简单的表函数逻辑。

****语法**：**

```

`` `sql
CREATE TABLES FUNCTION function_name(parameter_type1, parameter_type2, ...)
RETURNS ARRAY
PROPERTIES (
    "type" = "PYTHON_UDF",
    "symbol" = "function_name",
    "runtime_version" = "python_version",
    "always_nullable" = "true|false"
)
AS $$

```

```
def function_name(param1, param2, ...):
    '''函数说明'''
    # 处理逻辑
    yield result # 单列输出
    # 或
    yield (result1, result2, ...) # 多列输出
$$;
```

重要语法说明：

- 使用 CREATE TABLES FUNCTION（注意是 **TABLES**，复数形式）。
- 单列输出：ARRAY，如 ARRAY。
- 多列输出：ARRAY>。

示例 1：字符串分割（单列输出）

```
DROP FUNCTION IF EXISTS py_split(String, String);
```

```
CREATE TABLES FUNCTION py_split(String, String)
```

```
RETURNS ARRAY
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
    "symbol" = "split_string_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def split_string_udtf(text, delimiter):
```

```
    '''将字符串按分隔符分割为多行'''
```

```
    if text is not None and delimiter is not None:
```

```
        parts = text.split(delimiter)
```

```
        for part in parts:
```

```
            # 也支持 yield (part.strip(),)
```

```
            yield part.strip()
```

```
$$;
```

```
SELECT part
```

```
FROM (SELECT 'apple,banana,orange' as fruits) t
```

```
LATERAL VIEW py_split(fruits, ',') tmp AS part;
```

```
+-----+
| part |
+-----+
| apple |
| banana |
| orange |
+-----+
```

示例 2：生成数字序列（单列输出）

```
DROP FUNCTION IF EXISTS py_range(INT, INT);
```

```
CREATE TABLES FUNCTION py_range(INT, INT)
```

```
RETURNS ARRAY
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
    "symbol" = "generate_series_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def generate_series_udtf(start, end):
```

```
    '''生成从 start 到 end 的整数序列'''
```

```
    if start is not None and end is not None:
```

```
        for i in range(start, end + 1):
```

```
            yield i
```

```
$$;
```

```
SELECT num
```

```
FROM (SELECT 1 as start_val, 5 as end_val) t
```

```
LATERAL VIEW py_range(start_val, end_val) tmp AS num;
```

```
+-----+
| num |
+-----+
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
```

+-----+

```
SELECT date_add('2024-01-01', n) as date
FROM (SELECT 0 as start_val, 6 as end_val) t
LATERAL VIEW py_range(start_val, end_val) tmp AS n;
```

+-----+

date
2024-01-01
2024-01-02
2024-01-03
2024-01-04
2024-01-05
2024-01-06
2024-01-07

+-----+

示例 3：多列输出 (STRUCT)

```
DROP FUNCTION IF EXISTS py_duplicate(String, INT);
```

```
CREATE TABLES FUNCTION py_duplicate(String, INT)
```

```
RETURNS ARRAY<
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
    "symbol" = "duplicate_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def duplicate_udtf(text, n):
```

```
    """将文本复制 n 次，每次带序号"""
```

```
    if text is not None and n is not None:
```

```
        for i in range(n):
```

```
            yield (text, i + 1)
```

```
$$;
```

```
SELECT output, idx
```

```
FROM (SELECT 'Hello' as text, 3 as times) t
LATERAL VIEW py_duplicate(text, times) tmp AS output, idx;
```

```
+-----+-----+
| output | idx |
+-----+-----+
| Hello  |  1  |
| Hello  |  2  |
| Hello  |  3  |
+-----+-----+
```

示例 4：笛卡尔积（多列 STRUCT）

```
DROP FUNCTION IF EXISTS py_cartesian(String, String);
```

```
CREATE TABLES FUNCTION py_cartesian(String, String)
```

```
RETURNS ARRAY>
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
    "symbol" = "cartesian_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def cartesian_udtf(list1, list2):
```

```
    '''生成两个列表的笛卡尔积'''
```

```
    if list1 is not None and list2 is not None:
```

```
        items1 = [x.strip() for x in list1.split(',')]
        items2 = [y.strip() for y in list2.split(',')]
        for x in items1:
```

```
            for y in items2:
```

```
                yield (x, y)
```

```
    $$;
```

```
SELECT item1, item2
```

```
FROM (SELECT 'A,B' as list1, 'X,Y,Z' as list2) t
```

```
LATERAL VIEW py_cartesian(list1, list2) tmp AS item1, item2;
```

```
+-----+-----+
```


item1	item2
A	X
A	Y
A	Z
B	X
B	Y
B	Z

示例 5: JSON 数组解析

DROP FUNCTION IF EXISTS py_explode_json(String);

CREATE TABLE FUNCTION py_explode_json(String)

RETURNS ARRAY

PROPERTIES (

 "type" = "PYTHON_UDF",
 "symbol" = "explode_json_udtf",
 "runtime_version" = "3.10.12",
 "always_nullable" = "true",
 "volatility" = "immutable"

)

AS \$\$

```
def explode_json_udtf(json_str):
    '''解析 JSON 数组，每个元素输出一行'''
    if json_str is not None:
        try:
            data = json.loads(json_str)
            if isinstance(data, list):
                for item in data:
                    yield (str(item),)
        except:
            pass # 解析失败则跳过
    $$;
```

SELECT element

FROM (SELECT ["apple", "banana", "cherry"] as json_data) t

LATERAL **VIEW** py_explode_json(json_data) tmp **AS** element;

```
+-----+
| element |
+-----+
| apple  |
| banana |
| cherry |
+-----+
```

模块模式 (Module Mode)

模块模式适合复杂的表函数逻辑，需要将 Python 代码打包成 .zip 压缩包，并在函数创建时引用。

步骤 1：编写 Python 模块

创建 text_udtf.py 文件：

```
def split_lines_udtf(text):
    """按行分割文本"""
    if text:
        lines = text.split('\n')
        for line in lines:
            line = line.strip()
            if line: # 过滤空行
                yield (line,)

def extract_emails_udtf(text):
    """提取文本中的所有邮箱地址"""
    if text:
        email_pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
        emails = re.findall(email_pattern, text)
        for email in emails:
            yield (email,)

def parse_json_object_udtf(json_str):
    """解析 JSON 对象，输出键值对"""
    if json_str:
        try:
            data = json.loads(json_str)
```

```

    if isinstance(data, dict):
        for key, value in data.items():
            yield (key, str(value))
    except:
        pass

def expand_json_array_udtf(json_str):
    """展开 JSON 数组中的对象，输出结构化数据"""
    if json_str:
        try:
            data = json.loads(json_str)
            if isinstance(data, list):
                for item in data:
                    if isinstance(item, dict):
                        # 假设每个对象都有 id, name, score 字段
                        item_id = item.get('id')
                        name = item.get('name')
                        score = item.get('score')
                        yield (item_id, name, score)
        except:
            pass

def ngram_udtf(text, n):
    """生成 N-gram 词组"""
    if text and n and n > 0:
        words = text.split()
        for i in range(len(words) - n + 1):
            ngram = ' '.join(words[i:i+n])
            yield (ngram,)

```

步骤 2：打包 Python 模块

必须将 Python 文件打包成 .zip 格式（即使只有单个文件）：

```
zip text_udtf.zip text_udtf.py
```

步骤 3：设置 .zip 包的路径

通过 file 参数指定 .zip 包的路径：

- 部署方式：本地文件系统（file:// 协议）；写法："file" = "file:///path/to/text_udtf.zip"

- 部署方式：HTTP/HTTPS 远程下载（http:// 或 https:// 协议）；写法："file" = "http://example.com/udtf/text_udtf.zip"``"file" = "https://s3.amazonaws.com/bucket/text_udtf.zip"

:::caution 注意

- 使用远程下载方式时，需确保所有 BE 节点都能访问该 URL。
- 首次调用时会下载文件，可能有一定延迟。
- 文件会被缓存，后续调用无需重复下载。
- 使用 file:// 时，Doris 会在 CREATE TABLES FUNCTION 阶段从 FE 节点读取该包并计算 checksum，在执行阶段从 BE 节点读取该包。因此，同一个 .zip 文件必须以相同绝对路径存在于所有 FE 和 BE 节点，并且文件内容必须完全一致。如果文件只存在于 BE 节点，函数创建会失败；如果 FE 和 BE 上的文件内容不一致，执行时会因 checksum 不匹配失败。:::

步骤 4：设置 symbol 参数

在模块模式下，symbol 用于指定函数在 ZIP 包中的位置，格式为：

[package_name.]module_name.function_name

参数说明：

- package_name（可选）：ZIP 包内顶层 Python 包的名称。
- module_name（必填）：包含目标函数的 Python 模块文件名（不含 .py 后缀）。
- function_name（必填）：UDTF 函数名。

解析规则：

- Doris 会将 symbol 字符串按 . 分割：
 - 如果得到**两个**子字符串，分别为 module_name 和 function_name。
 - 如果得到**三个及以上**的子字符串，开头为 package_name，中间为 module_name，结尾为 function_name。

:::caution Warning 命名空间应具备唯一性，避免与 Python 标准库或常用第三方库同名，以避免因模块遮蔽导致的依赖冲突及运行时异常。:::

步骤 5：创建 UDTF 函数

```
DROP FUNCTION IF EXISTS py_split_lines(STRING);
DROP FUNCTION IF EXISTS py_extract_emails(STRING);
DROP FUNCTION IF EXISTS py_parse_json(STRING);
```

```
DROP FUNCTION IF EXISTS py_expand_json(String);  
DROP FUNCTION IF EXISTS py_ngram(String, Int);
```

```
CREATE TABLES FUNCTION py_split_lines(String)  
RETURNS ARRAY  
PROPERTIES (  
  "type" = "PYTHON_UDF",  
  "file" = "file:///path/to/text_udtf.zip",  
  "symbol" = "text_udtf.split_lines_udtf",  
  "runtime_version" = "3.10.12",  
  "always_nullable" = "true",  
  "volatility" = "immutable"  
);
```

```
CREATE TABLES FUNCTION py_extract_emails(String)  
RETURNS ARRAY  
PROPERTIES (  
  "type" = "PYTHON_UDF",  
  "file" = "file:///path/to/text_udtf.zip",  
  "symbol" = "text_udtf.extract_emails_udtf",  
  "runtime_version" = "3.10.12",  
  "always_nullable" = "true",  
  "volatility" = "immutable"  
);
```

```
CREATE TABLES FUNCTION py_parse_json(String)  
RETURNS ARRAY  
PROPERTIES (  
  "type" = "PYTHON_UDF",  
  "file" = "file:///path/to/text_udtf.zip",  
  "symbol" = "text_udtf.parse_json_object_udtf",  
  "runtime_version" = "3.10.12",  
  "always_nullable" = "true",  
  "volatility" = "immutable"  
);
```

```
CREATE TABLES FUNCTION py_expand_json(String)  
RETURNS ARRAY  
PROPERTIES (  
  "type" = "PYTHON_UDF",
```

```

"file" = "file:///path/to/text_udtf.zip",
"symbol" = "text_udtf.expand_json_array_udtf",
"runtime_version" = "3.10.12",
"always_nullable" = "true",
"volatility" = "immutable"
);

```

CREATE TABLES FUNCTION py_ngram(**STRING**, **INT**)

RETURNS **ARRAY**

PROPERTIES (

```

  "type" = "PYTHON_UDF",
  "file" = "file:///path/to/text_udtf.zip",
  "symbol" = "text_udtf.ngram_udtf",
  "runtime_version" = "3.10.12",
  "always_nullable" = "true",
  "volatility" = "immutable"
);

```

步骤 6：使用函数

SELECT line

FROM (**SELECT** 'Line 1\nLine 2\nLine 3' **as** text) t

LATERAL **VIEW** py_split_lines(text) tmp **AS** line;

```

+-----+
| line |
+-----+
| Line 1 |
| Line 2 |
| Line 3 |
+-----+

```

SELECT email

FROM (**SELECT** 'Contact us at support@example.com or sales@company.org' **as** content) t

LATERAL **VIEW** py_extract_emails(content) tmp **AS** email;

```

+-----+
| email |
+-----+
| support@example.com |

```

| sales@company.org |

+-----+

```
SELECT k, v
FROM (SELECT '{"name": "Alice", "age": "25"}' as json_data) t
LATERAL VIEW py_parse_json(json_data) tmp AS k, v;
```

+-----+

| k | v |

+-----+

| name | Alice |

| age | 25 |

+-----+

```
SELECT id, name, score
FROM (
  SELECT '["id": 1, "name": "Alice", "score": 95.5], {"id": 2, "name": "Bob", "score": 88.0}]'
  as data
) t
LATERAL VIEW py_expand_json(data) tmp AS id, name, score;
```

+-----+

| id | name | score |

+-----+

| 1 | Alice | 95.5 |

| 2 | Bob | 88 |

+-----+

```
SELECT ngram
FROM (SELECT 'Apache Doris is a fast database' as text) t
LATERAL VIEW py_ngram(text, 2) tmp AS ngram;
```

+-----+

| ngram |

+-----+

| Apache Doris |

| Doris is |

| is a |

| a fast |

| fast database |

+-----+

删除 Python UDF

-- 语法

DROP FUNCTION IF EXISTS function_name(parameter_types);

-- 示例

DROP FUNCTION IF EXISTS py_split(STRING, STRING);

DROP FUNCTION IF EXISTS py_range(INT, INT);

DROP FUNCTION IF EXISTS py_explode_json(STRING);

修改 Python UDF

Doris 不支持直接修改已有函数，需要先删除再重新创建：

DROP FUNCTION IF EXISTS py_split(STRING, STRING);

CREATE TABLES FUNCTION py_split(STRING, STRING) ...;

参数说明

CREATE TABLES FUNCTION 参数

- 参数：function_name；说明：函数名称，遵循 SQL 标识符命名规则
- 参数：parameter_types；说明：参数类型列表，如 INT、STRING、DOUBLE 等
- 参数：RETURNS ARRAY；说明：返回的数组类型，定义输出结构 • 单列：ARRAY • 多列：ARRAY>

PROPERTIES 参数

- 参数：type；是否必需：是；默认值：-；说明：固定为 "PYTHON_UDF"
- 参数：symbol；是否必需：是；默认值：-；说明：Python 函数名。 • **内联模式**：直接写函数名，如 "split_string_udtf" • **模块模式**：格式为 [package_name.]module_name.function_name
- 参数：file；是否必需：否；默认值：-；说明：Python .zip 包路径，仅模块模式需要。支持三种协议： • file:// —— 本地文件系统路径 • http:// —— HTTP 远程下载 • https:// —— HTTPS 远程下载
- 参数：runtime_version；是否必需：是；默认值：-；说明：Python 运行时版本，如 "3.10.12"
- 参数：always_nullable；是否必需：否；默认值：true；说明：是否总是返回可空结果

- 参数：volatility；是否必需：否；默认值：immutable；说明：函数易变性。可选值为 immutable、stable、volatile。immutable：相同输入始终产生相同输出。大部分确定性 UDF 建议设置为 immutable，这样计划优化效果更好。stable：在同一条 SQL 语句内结果稳定，但不同语句之间可能变化，类似 now()。这类函数会禁用 SQL Cache 和物化视图改写。volatile：每次调用结果都可能变化，类似 random()。这类函数会禁用 SQL Cache、物化视图改写以及很多优化器改写规则。

runtime_version 说明

- 必须填写 Python 版本的**完整版本号**，格式为 x.x.x 或 x.x.xx。
- Doris 会在配置的 Python 环境中查找匹配该版本的解释器。

数据类型映射

Python UDTF 使用与 Python UDF 完全相同的数据类型映射规则，包括整数、浮点、字符串、日期时间、Decimal、布尔、数组、STRUCT 等所有类型。

详细的数据类型映射关系请参考：数据类型映射。

NULL 值处理

- Doris 会将 SQL 中的 NULL 值映射为 Python 的 None。
- 在函数中，需要检查参数是否为 None。
- yield 产出的值可以包含 None，表示该列为 NULL。

实际应用场景

场景 1：CSV 数据解析

DROP FUNCTION IF EXISTS py_parse_csv(String);

CREATE TABLES FUNCTION py_parse_csv(String)

RETURNS **ARRAY**>

PROPERTIES (

```
"type" = "PYTHON_UDF",
"symbol" = "parse_csv_udtf",
"runtime_version" = "3.10.12",
"always_nullable" = "true",
"volatility" = "immutable"
```

)

AS \$\$

```
def parse_csv_udtf(csv_data):
```

```

'''解析 CSV 格式的多行数据'''
if csv_data is None:
    return
lines = csv_data.strip().split('\n')
for line in lines:
    parts = line.split(',')
    if len(parts) >= 3:
        name = parts[0].strip()
        age = int(parts[1].strip()) if parts[1].strip().isdigit() else None
        city = parts[2].strip()
        yield (name, age, city)
$$;

SELECT name, age, city
FROM (
    SELECT 'Alice,25,Beijing\nBob,30,Shanghai\nCharlie,28,Guangzhou' as data
) t
LATERAL VIEW py_parse_csv(data) tmp AS name, age, city;

```

```

+-----+-----+-----+
| name  | age  | city  |
+-----+-----+-----+
| Alice | 25   | Beijing |
| Bob   | 30   | Shanghai |
| Charlie | 28   | Guangzhou |
+-----+-----+-----+

```

场景 2：日期范围生成

```

DROP FUNCTION IF EXISTS py_date_range(String, String);

```

```

CREATE TABLE FUNCTION py_date_range(String, String)

```

```

RETURNS ARRAY

```

```

PROPERTIES (

```

```

    "type" = "PYTHON_UDF",
    "symbol" = "date_range_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"

```

```

)

```

```

AS $$

```

```
from datetime import datetime, timedelta
```

```
def date_range_udtf(start_date, end_date):  
    """生成日期范围"""  
    if start_date is None or end_date is None:  
        return  
    try:  
        start = datetime.strptime(start_date, '%Y-%m-%d')  
        end = datetime.strptime(end_date, '%Y-%m-%d')  
        current = start  
        while current < end:
```

```
    PROPERTIES (  
        "type" = "PYTHON_UDF",  
        "symbol" = "tokenize_udtf",  
        "runtime_version" = "3.10.12",  
        "always_nullable" = "true",  
        "volatility" = "immutable"  
    )  
    AS $$
```

```
def tokenize_udtf(text):  
    """将文本分词，输出单词和位置"""  
    if text is None:  
        return  
    # 使用正则提取单词  
    words = re.findall(r'\b\w+\b', text.lower())  
    for i, word in enumerate(words, 1):  
        if len(word) >= 2: # 过滤单字符  
            yield (word, i)  
    $$;
```

```
SELECT word, position  
FROM (SELECT 'Apache Doris is a fast OLAP database' as text) t  
LATERAL VIEW py_tokenize(text) tmp AS word, position;
```

```
+-----+-----+  
| word  | position |  
+-----+-----+  
| apache |    1 |  
| doris  |    2 |
```

is	3
fast	5
olap	6
database	7

+-----+-----+

场景 4: URL 参数解析

DROP FUNCTION IF EXISTS py_parse_url_params(String);

CREATE TABLES FUNCTION py_parse_url_params(String)

RETURNS ARRAY<

Properties (

 "type" = "PYTHON_UDF",

 "symbol" = "parse_url_params_udtf",

 "runtime_version" = "3.10.12",

 "always_nullable" = "true",

 "volatility" = "immutable"

)

AS \$\$

from urllib.parse import urlparse, parse_qs

def parse_url_params_udtf(url):

 """解析 URL 参数"""

if url **is None**:

return

 try:

 parsed = urlparse(url)

 params = parse_qs(parsed.query)

for key, values **in** params.items():

for value **in** values:

 yield (key, value)

except:

 pass

\$\$;

SELECT param_name, param_value

FROM (

SELECT 'https://example.com/page?id=123&category=tech&tag=python&tag=database'

as url

) t

```
LATERAL VIEW py_parse_url_params(url) tmp AS param_name, param_value;
```

```
+-----+-----+
| param_name | param_value |
+-----+-----+
| id      | 123      |
| category | tech     |
| tag      | python   |
| tag      | database |
+-----+-----+
```

场景 5: IP 范围展开

```
DROP FUNCTION IF EXISTS py_expand_ip_range(String, String);
```

```
CREATE TABLES FUNCTION py_expand_ip_range(String, String)
```

```
RETURNS ARRAY
```

```
PROPERTIES (
```

```
    "type" = "PYTHON_UDF",
    "symbol" = "expand_ip_range_udtf",
    "runtime_version" = "3.10.12",
    "always_nullable" = "true",
    "volatility" = "immutable"
```

```
)
```

```
AS $$
```

```
def expand_ip_range_udtf(start_ip, end_ip):
```

```
    """展开 IP 地址范围（仅支持最后一段）"""
```

```
    if start_ip is None or end_ip is None:
```

```
        return
```

```
    try:
```

```
        # 假设格式: 192.168.1.10 到 192.168.1.20
```

```
        start_parts = start_ip.split('.
```

```
        end_parts = end_ip.split('.
```

```
    if len(start_parts) == 4 and len(end_parts) == 4:
```

```
        # 只展开最后一段
```

```
        if start_parts[:3] == end_parts[:3]:
```

```
            prefix = '!.join(start_parts[:3])
```

```
            start_num = int(start_parts[3])
```

```
            end_num = int(end_parts[3])
```

```
            for i in range(start_num, end_num + 1):
```

```

        yield (f"{prefix}.{i}")
    except:
        pass
$$;

```

```

SELECT ip
FROM (SELECT '192.168.1.10' as start_ip, '192.168.1.15' as end_ip) t
LATERAL VIEW py_expand_ip_range(start_ip, end_ip) tmp AS ip;

```

```

+-----+
| ip      |
+-----+
| 192.168.1.10 |
| 192.168.1.11 |
| 192.168.1.12 |
| 192.168.1.13 |
| 192.168.1.14 |
| 192.168.1.15 |
+-----+

```

性能优化建议

1. 控制输出行数

- 对于可能产生大量输出的场景，设置合理的上限。
- 避免笛卡尔积爆炸。

2. 避免重复计算

如果需要多次使用同一个计算结果，预先计算：

不推荐

```

def bad_split_udtf(text):
    for i in range(len(text.split(','))): # 每次都 split
        parts = text.split(',')
        yield (parts[i],)

```

推荐

```

def good_split_udtf(text):
    parts = text.split(',') # 只 split 一次
    for part in parts:
        yield (part,)

```

3. 使用生成器表达式

利用 Python 的生成器特性，避免创建中间列表：

不推荐

```
def bad_filter_udtf(text, delimiter):
    parts = text.split(delimiter)
    filtered = [p.strip() for p in parts if p.strip()] # 创建列表
    for part in filtered:
        yield (part,)
```

推荐

```
def good_filter_udtf(text, delimiter):
    parts = text.split(delimiter)
    for part in parts:
        part = part.strip()
        if part: # 直接过滤
            yield (part,)
```

4. 避免访问外部资源

- 不要在 UDTF 中访问数据库、文件、网络。
- 所有处理应基于输入参数。

限制与注意事项

1. 无状态限制

- Python UDTF 是**无状态**的，每次函数调用独立处理一行。
- 不能在多次调用之间保留状态。
- 如果需要跨行聚合，应使用 UDAF。

2. 性能考虑

- Python UDTF 性能低于内置表函数。
- 适用于逻辑复杂但数据量适中的场景。
- 大数据量场景优先考虑优化或使用内置函数。

3. 输出类型固定

- RETURNS ARRAY 定义的类型是固定的。
- yield 产出的值必须与定义匹配。
- 单列：yield value 或 yield (value,); 多列：yield (value1, value2, ...)。

4. 函数命名

- 同一函数名在不同数据库中可重复定义。
- 调用时建议指定数据库名以避免歧义。

5. 环境一致性

- 所有 BE 节点的 Python 环境必须一致。
- 包括 Python 版本、依赖包版本、环境配置。

常见问题 FAQ

Q1: UDTF 和 UDF 的区别是什么?

A: **UDF** 输入单行，输出单行，为一对一关系。**UDTF** 输入单行，输出零行或多行，为一对多关系。

示例：

```
SELECT py_upper(name) FROM users;
```

```
SELECT tag FROM users LATERAL VIEW py_split(tags, ',') tmp AS tag;
```

Q2: 如何输出多列?

A: 多列输出使用 STRUCT 定义返回类型，并在 yield 时产出元组：

```
CREATE TABLE FUNCTION func(...)
```

```
RETURNS ARRAY>
```

```
...
```

```
def func(...):
```

```
    yield (123, 'hello') # 对应 col1 和 col2
```

Q3: 为什么我的 UDTF 没有输出?

A: 可能的原因：

1. **未调用 yield**：确保在函数中调用了 yield。
2. **条件过滤**：所有数据都被过滤掉了。
3. **异常被捕获**：检查是否有 try-except 吞掉了错误。
4. **NULL 输入**：输入是 NULL 且函数直接返回。

Q4: UDTF 可以维护状态吗?

A: 不能。Python UDTF 是无状态的，每次函数调用独立处理一行。如果需要跨行聚合或维护状态，应使用 Python UDAF。

Q5: 如何限制 UDTF 的输出行数?

A: 在函数中添加计数器或条件判断:

```
def limited_udtf(data):
    max_rows = 1000
    count = 0
    for item in data.split(','):
        if count >= max_rows:
            break
        yield (item,)
        count += 1
```

Q6: UDTF 输出的数据类型有限制吗?

A: UDTF 支持所有 Doris 数据类型，包括基本类型（INT、STRING、DOUBLE 等）和复杂类型（ARRAY、STRUCT、MAP 等）。输出类型必须在 RETURNS ARRAY 中明确定义。

Q7: 可以在 UDTF 中访问外部资源吗?

A: 技术上可以，但**强烈不推荐**。UDTF 应该是纯函数式的，只基于输入参数进行处理。访问外部资源（数据库、文件、网络）会导致性能问题和不可预测的行为。

Python UDF/UDAF/UDTF 环境配置与多版本管理

Python 环境管理

在使用 Python UDF/UDAF/UDTF 之前，请确保 Doris 的 Backend (BE) 节点已正确配置 Python 运行环境。Doris 支持通过 **Conda** 或 **Virtual Environment (venv)** 管理 Python 环境，允许不同的 UDF 使用不同版本的 Python 解释器和依赖库。

Doris 提供两种 Python 环境管理方式:

- **Conda 模式**: 使用 Miniconda/Anaconda 管理多版本环境。
- **Venv 模式**: 使用 Python 内置的虚拟环境 (venv) 管理多版本环境。

第三方库的安装与使用

Python UDF、UDAF、UDTF 都可以使用第三方库。但由于 Doris 的分布式特性，需要在**所有 BE 节点**上统一安装第三方库，否则会导致部分节点执行失败。

安装步骤

1. 在每个 BE 节点上安装依赖：

使用 pip 安装

```
pip install numpy pandas requests
```

或使用 conda 安装

```
conda install numpy pandas requests -y
```

2. 在函数中导入并使用：

在 UDF/UDAF/UDTF 函数中使用

```
def my_function(x):  
    return np.sqrt(x)
```

注意事项

- **pandas 和 pyarrow 是强制依赖**，必须在所有 Python 环境中预先安装，否则 Python UDF/UDAF/UDTF 无法运行。
- 必须在**所有 BE 节点**上安装相同版本的依赖，否则会导致部分节点执行失败。
- 安装路径要与对应 UDF/UDAF/UDTF 使用的 Python 运行时环境一致。
- 建议使用虚拟环境或 Conda 环境管理依赖，避免与系统 Python 环境冲突。

BE 配置参数

在所有 BE 节点的 be.conf 配置文件中设置以下参数，并**重启 BE** 使配置生效。

配置参数说明

- 参数名：enable_python_udf_support；类型：bool；可选值：true / false；默认值：false；说明：是否启用 Python UDF 功能
- 参数名：python_env_mode；类型：string；可选值：conda / venv；默认值：""；说明：Python 多版本环境管理方式
- 参数名：python_conda_root_path；类型：string；可选值：目录路径；默认值：""；说明：Miniconda 的根目录仅在 python_env_mode = conda 时生效

- 参数名：python_venv_root_path；类型：string；可选值：目录路径；默认值：\${DORIS_HOME}/lib/udf/python；说明：venv 多版本管理的根目录仅在 python_env_mode = venv 时生效
- 参数名：python_venv_interpreter_paths；类型：string；可选值：路径列表（用：分隔）；默认值：""；说明：可用 Python 解释器的目录列表仅在 python_env_mode = venv 时生效
- 参数名：max_python_process_num；类型：int32；可选值：整数；默认值：0；说明：Python Server 进程池最多运行的进程数 0 表示使用 CPU 核数作为默认值，用户可以设置其他正整数覆盖默认值

方式一：使用 Conda 管理 Python 环境

1. 配置 BE

在 be.conf 中添加以下配置：

```
## be.conf
enable_python_udf_support = true
python_env_mode = conda
python_conda_root_path = /path/to/miniconda3
```

2. 环境查找规则

Doris 会在 \${python_conda_root_path}/envs/ 目录下查找与 UDF 中 runtime_version 匹配的 Conda 环境。

匹配规则：

- runtime_version **必须填写 Python 版本的完整版本号**，格式为 x.x.x 或 x.x.xx，例如 "3.9.18"、"3.12.11"。
- Doris 会遍历所有 Conda 环境，检查每个环境中 Python 解释器的实际版本是否与 runtime_version 完全匹配。
- 如果找不到匹配的环境，则会报错：Python environment with version x.x.x not found。

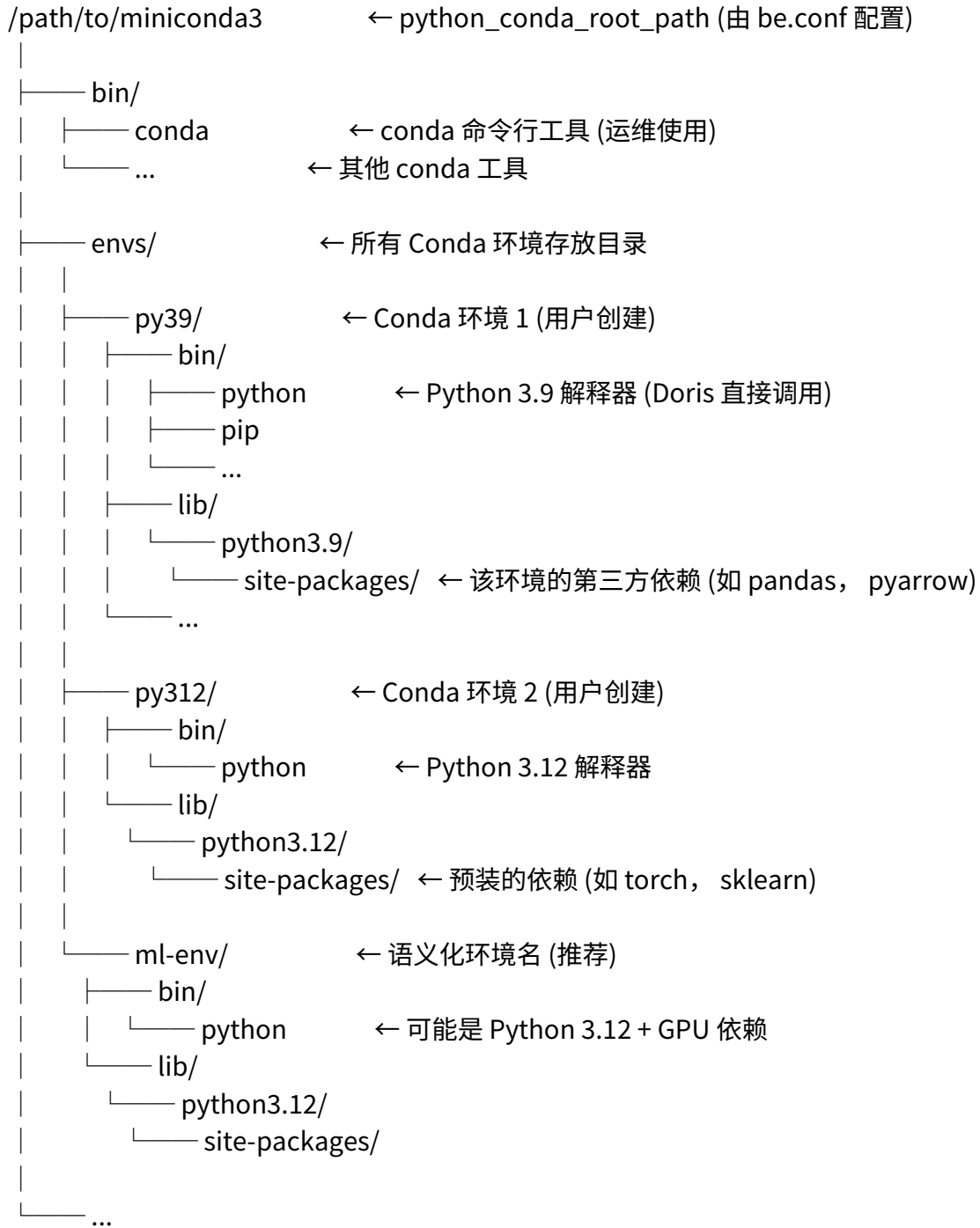
示例：

- UDF 中指定 runtime_version = "3.9.18"，Doris 会在所有环境中查找 Python 版本为 3.9.18 的环境。
- 环境名称可以是任意的（如 py39、my-env、data-science 等），只要该环境中的 Python 版本为 3.9.18 即可。

- 必须填写完整版本号，不能使用版本前缀，如 "3.9" 或 "3.12"。

3. 目录结构示意图

Doris BE 节点文件系统结构 (Conda 模式)



4. 创建 Conda 环境

:::caution 注意 Doris Python UDF/UDAF/UDTF 功能**强制依赖** pandas 和 pyarrow 两个库，**必须在所有 Python 环境中预先安装这两个依赖**，否则 UDF 将无法正常运行。 :::

在所有 BE 节点上执行以下命令创建 Python 环境：

安装 Miniconda (如果尚未安装)

wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3-latest-Linux-x86_64.sh -b -p /opt/miniconda3

创建 Python 3.9.18 环境并安装必需的依赖 (环境名可自定义)

/opt/miniconda3/bin/conda create -n py39 python=3.9.18 pandas pyarrow -y

创建 Python 3.12.11 环境并预装依赖 (重要: Python 版本必须精确指定,且必须安装 pandas 和 pyarrow)

/opt/miniconda3/bin/conda create -n py312 python=3.12.11 pandas pyarrow numpy -y

激活环境并安装额外依赖

source /opt/miniconda3/bin/activate py39

conda install requests beautifulsoup4 -y

conda deactivate

验证环境中的 Python 版本

/opt/miniconda3/envs/py39/bin/python --version *# 应输出: Python 3.9.18*

/opt/miniconda3/envs/py312/bin/python --version *# 应输出: Python 3.12.11*

5. 在 UDF 中使用

-- 使用 Python 3.12.11 环境

CREATE FUNCTION py_ml_predict(**DOUBLE**)

RETURNS **DOUBLE**

PROPERTIES (

 "type" = "PYTHON_UDF",

 "symbol" = "evaluate",

 "runtime_version" = "3.12.11", *-- 必须指定完整版本号, 匹配 Python 3.12.11*

 "always_nullable" = "true",

 "volatility" = "immutable"

)

AS \$\$

def evaluate(x):

可以使用 Python 3.12.11 环境中安装的库

```
return x * 2
$$;
```

-- 注意: 无论环境名是 `py312` 还是 `ml-env`, 只要 Python 版本是 3.12.11, 都可以使用
-- `runtime_version` 只关注 Python 版本, 不关注环境名称

方式二: 使用 Venv 管理 Python 环境

1. 配置 BE

在 `be.conf` 中添加以下配置:

```
## be.conf
enable_python_udf_support = true
python_env_mode = venv
python_venv_root_path = /doris/python_envs
python_venv_interpreter_paths =
/opt/python3.9/bin/python3.9:/opt/python3.12/bin/python3.12
```

2. 配置参数说明

- **python_venv_root_path**: 虚拟环境的根目录, 所有 `venv` 环境都将创建在此目录下。
- **python_venv_interpreter_paths**: 以英文冒号 : 分隔的 Python 解释器绝对路径列表。Doris 会检查每个解释器的版本, 并根据 UDF 中指定的 `runtime_version` (完整版本号, 如 "3.9.18") 匹配对应的解释器。

3. 目录结构示意图

```
## Doris BE 配置 (be.conf)
python_venv_interpreter_paths =
"/opt/python3.9/bin/python3.9:/opt/python3.12/bin/python3.12"
python_venv_root_path = /doris/python_envs
```

```
/opt/python3.9/bin/python3.9    ← 系统预装 Python 3.9
/opt/python3.12/bin/python3.12  ← 系统预装 Python 3.12
```

```
/doris/python_envs/             ← 所有虚拟环境的根目录 (python_venv_root_path)
|
|—— python3.9.18/               ← 环境 ID = Python 完整版本
|   |—— bin/
|   |   |—— python
|   |   |—— pip
```

```

|   └── lib/python3.9/site-packages/
|       ├── pandas==2.1.0
|       └── pyarrow==15.0.0
|
|── python3.12.11/           ← Python 3.12.11 环境
|   ├── bin/
|   │   ├── python
|   │   └── pip
|   └── lib/python3.12/site-packages/
|       ├── pandas==2.1.0
|       └── pyarrow==15.0.0
|
|── python3.12.10/         ← Python 3.12.10 环境
|   └── ...

```

4. 创建 Venv 环境

⚠️ 注意 Doris Python UDF/UDAF/UDTF 功能**强制依赖** pandas 和 pyarrow 两个库，**必须在所有 Python 环境中预先安装这两个依赖**，否则 UDF 将无法正常运行。 ⚠️

在所有 BE 节点上执行以下命令：

创建虚拟环境根目录

```
mkdir -p /doris/python_envs
```

使用 Python 3.9 创建虚拟环境

```
/opt/python3.9/bin/python3.9 -m venv /doris/python_envs/python3.9.18
```

激活环境并安装必需的依赖 (pandas 和 pyarrow 必须安装)

```
source /doris/python_envs/python3.9.18/bin/activate
```

```
pip install pandas pyarrow numpy
```

```
deactivate
```

使用 Python 3.12 创建虚拟环境

```
/opt/python3.12/bin/python3.12 -m venv /doris/python_envs/python3.12.11
```

激活环境并安装必需的依赖 (pandas 和 pyarrow 必须安装)

```
source /doris/python_envs/python3.12.11/bin/activate
```

```
pip install pandas pyarrow numpy scikit-learn
```

```
deactivate
```

5. 在 UDF 中使用

-- 使用 Python 3.9.18 环境

```
CREATE FUNCTION py_clean_text(STRING)  
RETURNS STRING  
PROPERTIES (  
    "type" = "PYTHON_UDF",  
    "symbol" = "evaluate",  
    "runtime_version" = "3.9.18", -- 必须指定完整版本号, 匹配 Python 3.9.18  
    "always_nullable" = "true",  
    "volatility" = "immutable"  
)  
AS $$  
def evaluate(text):  
    return text.strip().upper()  
$$;
```

-- 使用 Python 3.12.11 环境

```
CREATE FUNCTION py_calculate(DOUBLE)  
RETURNS DOUBLE  
PROPERTIES (  
    "type" = "PYTHON_UDF",  
    "symbol" = "evaluate",  
    "runtime_version" = "3.12.11", -- 必须指定完整版本号, 匹配 Python 3.12.11  
    "always_nullable" = "true",  
    "volatility" = "immutable"  
)  
AS $$  
  
def evaluate(x):  
    return np.sqrt(x)  
$$;
```

环境管理最佳实践

1. 选择合适的管理方式

- 场景：需要频繁切换 Python 版本；推荐方式：Conda；原因：环境隔离性好，依赖管理简单
- 场景：已有 Conda 环境；推荐方式：Conda；原因：可直接复用现有环境
- 场景：系统资源有限；推荐方式：Venv；原因：占用空间小，启动快
- 场景：已有 Python 系统环境；推荐方式：Venv；原因：无需额外安装 Conda

2. 环境一致性要求

:::caution 注意 所有 BE 节点必须配置**完全相同**的 Python 环境，包括：

- Python 版本必须一致
- 已安装的依赖包及其版本必须一致
- 环境目录路径必须一致 :::

注意事项

1. 配置修改生效

- 修改 be.conf 后，**必须重启 BE 进程**才能生效。
- 重启前请确保配置正确，避免服务中断。

2. 路径验证

配置前请确保路径正确：

Conda 模式: 验证 conda 路径

```
ls -la /opt/miniconda3/bin/conda  
/opt/miniconda3/bin/conda env list
```

Venv 模式: 验证解释器路径

```
/opt/python3.9/bin/python3.9 --version  
/opt/python3.12/bin/python3.12 --version
```

3. 权限设置

确保 Doris BE 进程有权限访问 Python 环境目录：

Conda 模式

```
chmod -R 755 /opt/miniconda3
```

Venv 模式

```
chmod -R 755 /doris/python_envs  
chown -R doris:doris /doris/python_envs # 假设 BE 进程用户为 doris
```

4. 资源限制

根据实际需求调整 Python 进程池参数：

```
## 确认使用 CPU 核数（推荐，max_python_process_num = 0）  
max_python_process_num = 0
```

```
## 高并发场景，手动指定进程数
max_python_process_num = 128
```

```
## 资源受限场景，限制进程数
max_python_process_num = 32
```

环境验证

在每个 BE 节点上验证环境是否正确

Conda 模式

```
/opt/miniconda3/envs/py39/bin/python --version
/opt/miniconda3/envs/py39/bin/python -c "import pandas; print(pandas.__version__)"
```

Venv 模式

```
/doris/python_envs/python3.9.18/bin/python --version
/doris/python_envs/python3.9.18/bin/python -c "import pandas;
print(pandas.__version__)"
```

展示所有 BE 共有的 Python 版本

```
SHOW PYTHON VERSIONS;
```

```
+-----+-----+-----+-----+-----+
| Version | EnvName | EnvType | BasePath      | ExecutablePath          |
+-----+-----+-----+-----+-----+
| 3.9.18 | py39    | conda   | path/to/miniconda | path/to/miniconda/envs/py39/bin/python |
+-----+-----+-----+-----+-----+
```

展示指定版本中已安装的依赖

使用 SHOW PYTHON PACKAGES IN " 来展示指定版本中已安装的依赖，如果各 BE 存在依赖不相同的情况，会列出不相同的部分。

```
SHOW PYTHON PACKAGES IN '3.9.18'
```

各 BE 依赖安装情况完全相同：

```
+-----+-----+
| Package   | Version |
+-----+-----+
| pyarrow   | 21.0.0  |
| Bottleneck | 1.4.2   |
| jieba     | 0.42.1  |
```

six	1.17.0	
wheel	0.45.1	
python-dateutil	2.9.0.post0	
tzdata	2025.3	
setuptools	80.9.0	
numpy	2.0.1	
psutil	7.0.0	
pandas	2.3.3	
mkl_random	1.2.8	
pip	25.3	
snownlp	0.12.3	
pytz	2025.2	
mkl_fft	1.3.11	
mkl-service	2.4.0	
numexpr	2.10.1	
+-----+		

各 BE 安装依赖情况不同:

+-----+			
Package	Version	Consistent	Backends
+-----+			
pyarrow	21.0.0	Yes	
Bottleneck	1.4.2	Yes	
six	1.17.0	Yes	
jieba	0.42.1	No	127.0.0.1:9660
wheel	0.45.1	Yes	
python-dateutil	2.9.0.post0	Yes	
tzdata	2025.3	Yes	
setuptools	80.9.0	Yes	
numpy	2.0.1	Yes	
psutil	7.0.0	No	127.0.0.1:9660
pandas	2.3.3	Yes	
mkl_random	1.2.8	Yes	
pip	26.0.1	No	127.0.0.1:9077
pip	25.3	No	127.0.0.1:9660
snownlp	0.12.3	No	127.0.0.1:9660
pytz	2025.2	Yes	
numexpr	2.10.1	Yes	
mkl-service	2.4.0	Yes	

mkl_fft	1.3.11	Yes		
+-----+-----+-----+-----+				

常见问题排查

Q1: UDF 调用时提示 "Python environment not found"

原因:

- runtime_version 指定的版本在系统中不存在。
- 环境路径配置不正确。

解决方案:

检查 Conda 环境列表

conda env list

检查 Venv 解释器是否存在

ls -la /opt/python3.9/bin/python3.9

检查 BE 配置

grep python /path/to/be.conf

Q2: UDF 调用时提示 "ModuleNotFoundError: No module named 'xxx'"

原因: Python 环境中未安装所需依赖包。

Q3: 不同 BE 节点执行结果不一致

原因: 各 BE 节点的 Python 环境或依赖版本不一致。

解决方案:

1. 检查所有节点的 Python 版本和依赖版本。
2. 验证所有节点环境一致性。
3. 统一使用 requirements.txt (pip) 或 environment.yml (Conda) 部署环境，常见用法示例：
 - 使用 requirements.txt (pip) :

在开发环境中导出依赖

pip freeze > requirements.txt

在 BE 节点上使用目标 Python 安装依赖

/path/to/python -m pip install -r requirements.txt

- 使用 environment.yml (Conda) :

导出依赖

```
conda env export --from-history -n py312 -f environment.yml
```

在 BE 节点上创建环境

```
conda env create -f environment.yml -n py312
```

或更新已有环境

```
conda env update -f environment.yml -n py312
```

:::caution 注意

- 必须确保 pandas 和 pyarrow 出现在依赖文件中，并在所有 BE 节点中安装相同版本。
- 安装时务必使用与 Doris 配置一致的 Python 解释器或 Conda 路径（例如 /opt/miniconda3/bin/conda 或指定的 venv 解释器）。
- 建议将依赖文件纳入版本控制或放入共享存储，由运维统一分发到所有 BE 节点。
- 更多参考：pip 官方文档，Conda 环境导出/导入说明。 :::

Q4: 修改 be.conf 后未生效

可能的原因：未重启 BE 进程。

使用限制

1. 性能考虑：

- Python UDF 性能低于内置函数，建议用于逻辑复杂但数据量不大的场景。
- 对于大数据量处理，优先考虑向量化模式。

2. 类型限制：

- 不支持 HLL、Bitmap 等特殊类型。

3. 环境隔离：

- 同一函数名在不同数据库中可重复定义。
- 调用时需指定数据库名（如 db.func()）以避免歧义。

4. 并发限制：

- Python UDF 通过进程池执行，并发数受 max_python_process_num 限制。
- 高并发场景需适当调大该参数。